IMRAN PASHA

AI Backend Engineer | Python, FastAPI, LLMs, RAG, PostgreSQL

Hyderabad, India | +91 98445 48537 | imranpasha.ahmed@gmail.com | linkedin.com/in/imran-pasha-019b2b213 | github.com/Imranpasha30 | imranportfolio.duckdns.org

PROFESSIONAL SUMMARY

Backend engineer with nearly two years building production Python services and shipping Generative AI inside them. Founding backend engineer on Kaizer X, an AI video-production SaaS live at kaizerx.com, where I built a provider-agnostic LLM layer over Gemini, Claude and OpenAI with per-model rate limiting, circuit breakers and cost metering. Separately built a retrieval-augmented chatbot end to end on a Chroma vector store with local sentence-transformers embeddings, then attacked it against the OWASP LLM Top 10 and proved each defence with measurable deltas.

TECHNICAL SKILLS

Languages: Python 3.12, JavaScript (Node.js 20), TypeScript, SQL

Backend: FastAPI, Flask, Django, Express.js, SQLAlchemy 2.0 (async), Pydantic v2, Alembic, OpenAPI, object-oriented design, modular and reusable services

Generative AI: RAG pipelines, embedding models (sentence-transformers), vector databases (Chroma), chunking and retrieval, multi-provider LLM orchestration (Gemini, Claude, OpenAI, Ollama), structured-output validation, circuit breakers, cost metering, response caching, Deepgram and Whisper ASR, prompt-injection defence, OWASP LLM Top 10

Data & Caching: PostgreSQL 17 (composite indexing, query-plan tuning, advisory locks, FOR UPDATE SKIP LOCKED), Redis 7 (Streams, consumer groups, Lua scripting), SQLite, Supabase

Architecture: REST API design, microservices, event-driven workflows, message queues, WebSockets, Server-Sent Events, asyncio, background workers, idempotency keys

Cloud & DevOps: AWS (EC2, S3, Lambda), Cloudflare R2, Docker, NGINX, Railway, Render, GitHub Actions CI/CD, Linux, Prometheus, OpenTelemetry, pytest

Security: OAuth 2.0, JWT, RBAC, OWASP Top 10 and API Security Top 10, threat modeling, secure code review, SSRF/IDOR/BOLA mitigation, Burp Suite

PROFESSIONAL EXPERIENCE

Backend Engineer | Sharkify Private Limited — Dec 2024 – Present
Hyderabad, India

Kaizer X — AI video production and publishing SaaS — Apr 2026 – Present
Python 3.12, FastAPI, SQLAlchemy 2.0, PostgreSQL 17, Redis 7, Gemini, Claude, OpenAI, Deepgram, FFmpeg/NVENC, Docker

- Founded the backend, rewriting a Django prototype as **FastAPI and SQLAlchemy** and owning it solo for five months; it now serves 765 REST and WebSocket routes over 150 PostgreSQL tables.
- Built a **provider-agnostic LLM layer** over Gemini, Claude and OpenAI behind one request contract: per-model token-bucket rate limiting, per-provider circuit breakers that recover through a half-open probe, Retry-After aware retries and a per-job cost meter.
- Enforced **structured-output contracts** with Pydantic schema validation and a self-repair path, so malformed model responses are corrected rather than crashing a multi-stage pipeline.
- Implemented dual-engine speech-to-text over **Deepgram Nova-3 and Gemini** with automatic failover for Indic languages, overlap de-duplication and speaker-label voting.
- Cut repeat model spend with a **cross-pod Redis response cache** keyed on content hash, preset and language, tiered through disk, with hit, miss and bytes-saved counters on an admin dashboard.
- Made publishing safe to retry with **SHA-256 idempotency keys** arbitrated by a database UNIQUE constraint, over a PostgreSQL job queue using FOR UPDATE SKIP LOCKED with lease fencing and jittered backoff.
- Debugged a production queue runaway that grew one Redis stream from 8 to **619,000 entries**: traced it from the logs to a time-of-check-to-time-of-use race between two workers, fixed it with an atomic conditional claim, and wrote a regression test that reproduces it.

LLM Red-Team Platform — personal project | github.com/Imranpasha30/llm-redteam-platform Sep 2026

Python, Chroma, sentence-transformers, Flask, Ollama, OpenAI, Anthropic, promptfoo

- Built a **RAG application end to end**: document ingestion and chunking, local sentence-transformers embeddings, a persistent **Chroma vector store**, top-K retrieval preserving source metadata, and a Flask chat UI, with Ollama, OpenAI and Anthropic backends behind one interface.
- Wrapped it in an **OWASP LLM Top 10** harness mapping probes across LLM01 to LLM09, adding custom Python checks for supply chain and data poisoning, the two categories promptfoo cannot fully automate.
- Wrote the guardrail layer: prompt-injection scanning with **Unicode normalisation** that strips zero-width characters so evasion still matches, plus output scanning that redacts API keys, passwords, ID numbers, emails and phone numbers.
- Proved every defence with **measurable before-and-after deltas** behind one environment flag, using a planted canary key so a successful exfiltration is verifiable in a diff.

Voter Connect — campaign platform (live on Google Play) Dec 2025 – Jun 2026

FastAPI, PostgreSQL, Redis, async I/O, SQLAlchemy, JWT, Docker, GitHub Actions

- Built and tuned a backend over a **9.9M-row PostgreSQL** dataset, holding sub-second API latency through composite indexing, query-plan analysis and Redis caching of hot read paths.
- Implemented async I/O, JWT authentication and rate limiting, designed geolocation-verified endpoints separating administrator from volunteer authorization, and delivered CI/CD on GitHub Actions.

Editor Dashboard and KaizerFit — FastAPI services Jun 2025 – Oct 2025

FastAPI, async SQLAlchemy, asyncpg, PostgreSQL, Alembic, Server-Sent Events, Telegram Bot API, Docker, AWS EC2

- Built a newsroom submission CMS as principal author (98% of the code by git blame): 54 endpoints, revocable JWTs, and real-time dashboards without polling via PL/pgSQL triggers, pg_notify and **Server-Sent Events**.
- Led a fitness backend (47 endpoints, 21 tables) whose **offline-first sync engine** ingests five record types in one atomic bulk transaction, with ID mapping and hash-based deduplication so client retries never double-count.

Ozone Wash — field-service booking and payments platform Mar 2026 – Oct 2026

Node.js 20, Express 4, PostgreSQL (Supabase), Cloudflare R2, Razorpay, PayU, Easebuzz, node-cron

- Architected and built the **Node.js/Express** backend as sole backend engineer for six months: 23 modules, roughly 200 REST endpoints and 37 transactional SQL migrations defining 68 tables.
- Integrated three payment gateways behind one facade with raw-body **HMAC-SHA256 webhook verification**, replay deduplication and idempotent settlement, so a redelivered webhook never double-confirms a booking.

SECURITY ENGINEERING APPLIED TO AI AND BACKEND

Cyber security training applied directly to the systems above, including the AI ones.

- Treat **retrieved documents as untrusted input** in RAG systems: the moment corpus text enters a prompt it can carry adversarial instructions, so retrieval filters by document sensitivity before assembly and output is scanned before it reaches the caller.
- Closed a two-round adversarial security review on Kaizer X with a regression test behind every fix: cross-tenant authorization (**BOLA/IDOR**) on live-broadcast controls and WebSockets, OAuth-callback cross-site scripting, and password-reset links trusting an attacker-controlled Origin header.
- Applies **OWASP API Security Top 10** and the **OWASP LLM Top 10**, STRIDE threat modeling and secure code review from an IIT Roorkee programme and 49 hands-on lab rooms.

EDUCATION

B.Tech, Computer Science | Sahyadri College of Engineering (VTU), Mangalore 2020 – 2024

CERTIFICATIONS

Executive Post Graduate Certification in Cyber Security and Ethical Hacking | IIT Roorkee May 2026

iHUB DivyaSampark, IIT Roorkee with Intellipaat. Credential ID IPTIH26050044. 49 hands-on lab rooms covering web application testing, Burp Suite, Nmap, Metasploit, SQL injection, XSS and IDOR.

SELECTED ACHIEVEMENTS

- Shipped Generative AI into production rather than demos: multi-provider LLM orchestration with failover, cost metering and response caching serving a live SaaS.
- Built and then attacked my own RAG system against the OWASP LLM Top 10, proving each guardrail with measurable before-and-after deltas.